

## 8장. CSS 심화: 전환, 애니메이션, 변수

**학습 목표** - 의사 클래스와 의사 요소로 상태 기반 스타일과 장식을 추가할 수 있습니다  
- transition으로 부드러운 상태 변화를 구현할 수 있습니다 - transform과 keyframes  
애니메이션으로 움직이는 효과를 만들 수 있습니다 - CSS 변수로 테마를 일관되게 관리할 수 있습니다

### 8.1 의사 클래스와 의사 요소

가짜

#### 도입

지금까지 배운 선택자는 요소의 이름, 클래스, 아이디로 대상을 골랐습니다. 그런데 "마우스를 올렸을 때만 색을 바꾸고 싶다"거나 "홀수 번째 항목에만 배경을 다르게 하고 싶다"면 어떻게 할까요? 이럴 때 사용하는 것이 **의사 클래스(pseudo-class)**와 **의사 요소(pseudo-element)**입니다. JavaScript 없이도 사용자 상호작용에 반응하는 스타일을 만들 수 있습니다.

#### 핵심 개념 설명

##### 의사 클래스 (Pseudo-class)

**의사 클래스**는 요소의 특정 **상태(state)**를 조건으로 스타일을 적용하는 선택자입니다. **콜론(:)** 하나를 앞에 붙여 사용합니다.

요소 자체가 변한 것이 아니라, 마우스가 올라왔다거나 포커스를 받았다거나 하는 **'상황'**에 반응하는 것입니다. 마치 신호등이 상황에 따라 색을 바꾸는 것과 같습니다.

##### 자주 사용하는 상태 의사 클래스

| 의사 클래스                 | 적용 조건                   |
|------------------------|-------------------------|
| <code>:hover</code>    | 마우스 커서를 요소 위에 올렸을 때     |
| <code>:focus</code>    | 키보드 탭이나 클릭으로 포커스를 받았을 때 |
| <code>:active</code>   | 마우스 버튼을 누르고 있는 순간       |
| <code>:visited</code>  | 사용자가 이미 방문한 링크          |
| <code>:disabled</code> | 비활성화된 폼 요소              |
| <code>:checked</code>  | 체크박스·라디오 버튼이 선택된 상태     |

## 구조 의사 클래스 (Structural Pseudo-class)

구조 의사 클래스는 DOM 트리에서 요소의 위치를 기준으로 선택합니다.

| 의사 클래스                        | 의미                                                           |
|-------------------------------|--------------------------------------------------------------|
| <code>:first-child</code>     | 부모의 첫 번째 자식                                                  |
| <code>:last-child</code>      | 부모의 마지막 자식                                                   |
| <code>:nth-child(n)</code>    | 부모의 n번째 자식 ( <code>2n</code> 이면 짝수, <code>2n+1</code> 이면 홀수) |
| <code>:nth-child(odd)</code>  | 홀수 번째 자식 ( <code>:nth-child(2n+1)</code> 과 동일)               |
| <code>:nth-child(even)</code> | 짝수 번째 자식 ( <code>:nth-child(2n)</code> 과 동일)                 |
| <code>:not(선택자)</code>        | 괄호 안 선택자에 해당하지 않는 요소                                         |

## 의사 요소 (Pseudo-element)

의사 요소는 HTML에 실제로 존재하지 않지만 CSS로 만들어 낼 수 있는 가상의 요소입니다.

콜론 두 개( `::` )를 사용합니다.

| 의사 요소                       | 역할                         |
|-----------------------------|----------------------------|
| <code>::before</code>       | 요소의 콘텐츠 <b>앞에</b> 가상 요소 삽입 |
| <code>::after</code>        | 요소의 콘텐츠 <b>뒤에</b> 가상 요소 삽입 |
| <code>::first-line</code>   | 블록 요소의 첫 번째 줄              |
| <code>::first-letter</code> | 블록 요소의 첫 번째 글자             |
| <code>::selection</code>    | 사용자가 드래그로 선택한 텍스트 영역       |

`::before` 와 `::after` 는 반드시 `content` 속성을 가져야 화면에 나타납니다. `content: ""` 처럼 빈 문자열도 가능합니다.

## 코드로 확인하기

### `:hover`, `:focus`, `:active` 상태 스타일링

버튼에 마우스 상태별 다른 색상을 적용합니다. **마우스를 올리면(hover) 배경색이 바뀌고, 클릭 순간(active)은 더 진해집니다.**

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>의사 클래스 예제</title>
  <style>
    /* 기본 버튼 스타일 */
    .btn {
      display: inline-block;
      padding: 12px 28px;
      background-color: #3b82f6;
      color: #fff;
      border: none;
      border-radius: 6px;
      font-size: 1rem;
      cursor: pointer;
      outline: none;
    }

    /* 마우스를 올렸을 때 */
    .btn:hover {
      background-color: #2563eb;
    }

    /* 키보드 포커스를 받았을 때 (접근성) */
    .btn:focus {
      outline: 3px solid #93c5fd;
      outline-offset: 2px;
    }

    /* 클릭하는 순간 */
    .btn:active {
      background-color: #1d4ed8;
    }

    /* 비활성화 상태 */
    .btn:disabled {
      background-color: #94a3b8;
      cursor: not-allowed;
    }
  </style>
</head>
<body>
  <button class="btn">기본 버튼</button>
  <button class="btn" disabled>비활성 버튼</button>
</body>
</html>
```

```
</body>
</html>
```

## 브라우저 렌더링 결과

- 파란 배경의 "기본 버튼"이 표시됩니다.
- 마우스를 올리면 배경이 더 진한 파란색으로 바뀝니다.
- 클릭하는 순간은 가장 진한 파란색이 됩니다.
- Tab 키로 포커스를 이동하면 하늘색 아웃라인이 나타납니다.
- "비활성 버튼"은 회색 배경에 커서가 금지 아이콘으로 바뀝니다.

### 줄별 코드 설명

| 줄  | 코드                                      | 설명                                                                                      |
|----|-----------------------------------------|-----------------------------------------------------------------------------------------|
| 17 | <code>.btn:hover</code>                 | <code>.btn</code> 클래스를 가진 요소에 마우스가 올라간 상태를 선택합니다                                        |
| 22 | <code>.btn:focus</code>                 | 키보드 탭이나 클릭으로 포커스를 받은 상태입니다. 접근성을 위해 반드시 시각적으로 구분해야 합니다                                  |
| 23 | <code>outline: 3px solid #93c5fd</code> | <code>:focus</code> 상태에서 테두리 바깥에 아웃라인을 그립니다. <code>border</code> 와 달리 레이아웃에 영향을 주지 않습니다 |
| 24 | <code>outline-offset: 2px</code>        | 아웃라인을 요소 테두리에서 2px 떨어뜨립니다                                                               |
| 28 | <code>.btn:active</code>                | 마우스 버튼을 누르고 있는 짧은 순간에만 적용됩니다                                                            |
| 33 | <code>.btn:disabled</code>              | HTML의 <code>disabled</code> 속성이 있는 요소를 선택합니다                                            |
| 34 | <code>cursor: not-allowed</code>        | 마우스 커서를 금지 아이콘으로 바꿔 비활성 상태를 시각적으로 전달합니다                                                 |

**베스트 프랙티스:** `:focus` 스타일은 절대 `outline: none` 으로 제거하지 않습니다. 키보드 사용자는 포커스 표시로 현재 위치를 파악하기 때문입니다. 시각적으로 어울리지 않는다면 `outline` 을 제거하는 대신 `box-shadow` 나 `border` 로 대체 스타일을 반드시 제공합니다.

---

## :nth-child로 줄무늬 표 만들기

`nth-child`로 짝수 행에만 배경색을 입혀 읽기 편한 줄무늬 표를 만듭니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>구조 의사 클래스</title>
  <style>
    table {
      border-collapse: collapse;
      width: 100%;
    }

    th, td {
      padding: 10px 16px;
      text-align: left;
      border-bottom: 1px solid #e2e8f0;
    }

    thead th {
      background-color: #1e40af;
      color: #fff;
    }

    /* 짝수 번째 tbody 행에 배경색 */
    tbody tr:nth-child(even) {
      background-color: #f1f5f9;
    }

    /* 마지막 행 아래 테두리 제거 */
    tbody tr:last-child td {
      border-bottom: none;
    }

    /* hover 시 강조 */
    tbody tr:hover {
      background-color: #dbeafe;
    }
  </style>
</head>
<body>
  <table>
    <thead>
      <tr>
        <th>이름</th>
        <th>역할</th>
        <th>경험</th>
      </tr>
    </thead>
    <tbody>
      <tr><td>김민준</td><td>디자이너</td><td>3년</td></tr>

```

```

<tr><td>이 서연</td><td>개발자</td><td>5년</td></tr>
<tr><td>박지호</td><td>기획자</td><td>2년</td></tr>
<tr><td>최다은</td><td>개발자</td><td>4년</td></tr>
</tbody>
</table>
</body>
</html>

```

## 브라우저 렌더링 결과

- 헤더 행은 진한 파란 배경에 흰 글자로 표시됩니다.
- 짝수 행(2번째, 4번째)에 연한 회색 배경이 적용됩니다.
- 마우스를 올린 행은 연한 파란색으로 강조됩니다.

### 줄별 코드 설명

| 줄  | 코드                                    | 설명                                                                                                                           |
|----|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 22 | <code>tbody tr:nth-child(even)</code> | <code>tbody</code> 안의 <code>tr</code> 중 짝수 번째(2, 4, 6번째...)를 선택합니다. <code>odd</code> 는 홀수, <code>2n+1</code> 처럼 수식도 사용 가능합니다 |
| 27 | <code>tbody tr:last-child td</code>   | <code>tbody</code> 의 마지막 <code>tr</code> 안의 <code>td</code> 를 선택합니다. 결합 선택자와 의사 클래스를 조합한 예입니다                                |

## ::before와 ::after로 장식 추가하기

`::before`와 `::after`로 HTML을 수정하지 않고 시각적 장식 요소를 추가합니다.



```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>의사 요소 예제</title>
  <style>
    /* 필수 항목 표시(*) */
    .required::after {
      content: " *";
      color: #ef4444;
      font-weight: bold;
    }

    /* 아이콘 없이 인용 블록 꾸미기 */
    blockquote {
      position: relative;
      padding: 16px 16px 16px 48px;
      background-color: #f8fafc;
      border-left: 4px solid #3b82f6;
      margin: 24px 0;
    }

    blockquote::before {
      content: "\201C"; /* 여는 큰따옴표 유니코드 */
      position: absolute;
      left: 12px;
      top: 8px;
      font-size: 3rem;
      line-height: 1;
      color: #3b82f6;
      font-family: Georgia, serif;
    }

    /* 호버 밑줄 효과 */
    .underline-hover {
      position: relative;
      text-decoration: none;
      color: #1e40af;
    }

    .underline-hover::after {
      content: "";
      position: absolute;
      bottom: -2px;
      left: 0;
      width: 0;          /* 초기 너비 0 */
      height: 2px;
      background-color: #3b82f6;
      transition: width 0.3s ease;
    }
  </style>

```

```

    }

    .underline-hover:hover::after {
        width: 100%;    /* hover 시 전체 너비로 확장 */
    }
</style>
</head>
<body>
    <form>
        <label class="required">이름</label>
        <input type="text">
    </form>

    <blockquote>
        디자인은 어떻게 보이는지가 아니라 어떻게 작동하는지에 관한 것입니다.
    </blockquote>

    <a href="#" class="underline-hover">마우스를 올려보세요</a>
</body>
</html>

```

## 브라우저 렌더링 결과

- "이름" 레이블 뒤에 빨간 별표(\*)가 자동으로 표시됩니다.
- 인용 블록 왼쪽에 큰 파란 따옴표 장식이 나타납니다.
- 링크 텍스트에 마우스를 올리면 파란 밑줄이 왼쪽에서 오른쪽으로 펼쳐집니다.

## 줄별 코드 설명

| 줄  | 코드                                         | 설명                                                                                |
|----|--------------------------------------------|-----------------------------------------------------------------------------------|
| 4  | <code>.required::after</code>              | <code>.required</code> 클래스 요소의 콘텐츠 뒤에 가상 요소를 만듭니다                                 |
| 5  | <code>content: " *"</code>                 | <code>::after</code> 에 반드시 있어야 하는 속성입니다. 표시할 내용을 문자열로 지정합니다                       |
| 27 | <code>content: "\201C"</code>              | CSS에서 유니코드 문자를 <code>\</code> + 코드 포인트로 표현합니다. U+201C는 여는 큰따옴표입니다                 |
| 29 | <code>position: absolute</code>            | <code>blockquote</code> 에 <code>position: relative</code> 가 있어 그 안에서 절대 위치로 배치됩니다 |
| 43 | <code>content: ""</code>                   | 내용이 없는 빈 가상 요소를 만듭니다. 순수 시각 장식에 사용합니다                                             |
| 46 | <code>width: 0</code>                      | 초기 너비를 0으로 설정하여 처음에는 밑줄이 보이지 않게 합니다                                               |
| 50 | <code>transition: width 0.3s ease</code>   | <code>width</code> 속성 변화를 0.3초 동안 부드럽게 처리합니다 (transition은 다음 절에서 자세히 배웁니다)        |
| 54 | <code>.underline-hover:hover::after</code> | 의사 클래스( <code>:hover</code> )와 의사 요소( <code>::after</code> )를 함께 사용한 고급 선택자입니다    |

**주의:** `::before` 와 `::after` 로 삽입한 내용은 스크린 리더(Screen Reader)에서 읽힐 수 있습니다. 순수하게 장식 목적이라면 `content: ""` 를 사용하고, `aria-hidden` 을 고려합니다. 반드시 읽혀야 하는 의미 있는 내용은 실제 HTML로 작성합니다.

## 절 요약

- 의사 클래스( `:` )는 요소의 상태나 위치에 따라 스타일을 적용합니다.
- `:hover` , `:focus` , `:active` 는 사용자 상호작용을 CSS만으로 표현합니다.
- `:nth-child(n)` 는 수식으로 특정 순번의 요소를 선택합니다.
- 의사 요소( `::` )는 HTML을 수정하지 않고 가상의 콘텐츠나 장식을 추가합니다.
- `::before` 와 `::after` 는 반드시 `content` 속성이 있어야 합니다.

## 8.2 transition으로 부드러운 변화 주기

### 도입

버튼에 마우스를 올렸을 때 색이 순식간에 바뀌면 어색해 보입니다. 전등 스위치를 켤 때 즉시 켜지는 대신 서서히 밝아지는 디머 스위치처럼, **전환(transition)**은 CSS 속성 값이 변할 때 그 변화를 지정한 시간 동안 부드럽게 보간합니다. JavaScript 없이 자연스러운 인터랙션 효과를 만들 수 있습니다.

---

### 핵심 개념 설명

#### 전환 (CSS Transition)

`transition` 은 속성 값이 **A에서 B로 변할 때** 그 중간 과정을 시각적으로 표현합니다. 변화의 시작은 항상 외부 트리거( `:hover` , `:focus` , JavaScript의 클래스 추가 등)에 의해 일어납니다.

전환은 네 가지 구성 요소를 가집니다.

`transition`

---

| 항목    | 내용                                                                                                                    |
|-------|-----------------------------------------------------------------------------------------------------------------------|
| 설명    | CSS 속성의 변화를 지정한 시간 동안 부드럽게 보 간하는 단축 속성입니다                                                                             |
| 매개변수  | <code>transition-property</code> : 전환을 적용할 속성 이름 (기본값: <code>all</code> )                                             |
|       | <code>transition-duration</code> : 전환 시간. 필수 값이 며 <code>s</code> 또는 <code>ms</code> 단위를 사용합니다 (예: <code>0.3s</code> ) |
|       | <code>transition-timing-function</code> : 가속/감속 곡선. 기본값 <code>ease</code>                                             |
|       | <code>transition-delay</code> : 전환 시작 전 대기 시간. 기본값 <code>0s</code>                                                    |
| 문법    | <code>transition: property duration timing-function delay</code>                                                      |
| 사용 예시 | <code>transition: background-color 0.3s ease 0s</code>                                                                |

### 타이밍 함수 (Timing Function, Easing)

타이밍 함수는 시간에 따른 속성 변화의 **속도 곡선**을 정의합니다. 같은 지속 시간이라도 곡선에 따라 움직임의 느낌이 달라집니다.

*CSS transition 타이밍 함수(Easing) 비교 그래프*

| 값                                      | 특성                 | 사용 상황                        |
|----------------------------------------|--------------------|------------------------------|
| <code>ease</code>                      | 느리게 시작, 빠르게, 느리게 끝 | 기본값. 대부분의 상황에 자연스럽게 다        |
| <code>linear</code>                    | 일정한 속도             | 회전하는 로딩 스피너처럼 균일한 속도가 필요할 때  |
| <code>ease-in</code>                   | 느리게 시작, 빠르게 끝      | 화면 밖으로 사라지는 요소               |
| <code>ease-out</code>                  | 빠르게 시작, 느리게 끝      | 화면 안으로 들어오는 요소 (가장 자연스러운 감속) |
| <code>ease-in-out</code>               | 느리게 시작, 빠르게, 느리게 끝 | ease보다 좌우 대칭이 더 뚜렷한 S자 곡선    |
| <code>cubic-bezier(x1,y1,x2,y2)</code> | 완전한 커스텀 곡선         | 세밀한 조정이 필요할 때                |

## transition을 적용할 수 있는 속성

숫자로 보간 가능한 속성에만 transition이 작동합니다.

- 가능: `color`, `background-color`, `opacity`, `width`, `height`, `margin`, `padding`, `transform`, `font-size` 등
- 불가능: `display`, `visibility` (단, opacity는 가능), `font-family` 등

## 코드로 확인하기

### 기본 transition 적용

버튼에 hover 시 배경색, 크기, 그림자가 부드럽게 전환되는 효과를 만듭니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>transition 기본</title>
  <style>
    .card {
      width: 200px;
      padding: 24px;
      background-color: #fff;
      border: 1px solid #e2e8f0;
      border-radius: 8px;
      text-align: center;
      cursor: pointer;

      /* 여러 속성에 각각 다른 전환 지정 */
      transition:
        background-color 0.3s ease,
        box-shadow 0.3s ease,
        transform 0.2s ease-out;
    }

    .card:hover {
      background-color: #eff6ff;
      box-shadow: 0 8px 24px rgba(59, 130, 246, 0.2);
      transform: translateY(-4px); /* transform은 다음 절에서 자세히 다룹니다 */
    }

    /* 단일 속성 전환 예시 */
    .progress-bar {
      height: 12px;
      background-color: #e2e8f0;
      border-radius: 6px;
      margin-top: 24px;
      overflow: hidden;
    }

    .progress-fill {
      height: 100%;
      width: 30%;
      background-color: #3b82f6;
      border-radius: 6px;
      transition: width 1s ease-in-out;
    }

    .progress-bar:hover .progress-fill {
      width: 80%;
    }
  </style>

```

```

</head>
<body>
  <div class="card">
    <p>카드 위에 마우스를 올려보세요</p>
  </div>

  <div class="progress-bar">
    <div class="progress-fill"></div>
  </div>
</body>
</html>

```

## 브라우저 렌더링 결과

- 카드: 마우스를 올리면 배경색이 연한 파란색으로, 그림자가 파란 계열로 부드럽게 전환됩니다. 동시에 카드가 4px 위로 떠오릅니다.
- 프로그레스 바: 마우스를 올리면 파란 막대가 30%에서 80%까지 1초에 걸쳐 늘어납니다.

### 줄별 코드 설명

| 줄  | 코드                                             | 설명                                                                                                     |
|----|------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 14 | <code>transition:</code>                       | 여러 속성에 개별 전환을 지정할 때 쉼표로 구분합니다                                                                          |
| 15 | <code>background-color 0.3s ease,</code>       | <code>background-color</code> 속성을 0.3초, ease 곡선으로 전환합니다                                                |
| 17 | <code>transform 0.2s ease-out</code>           | <code>transform</code> 은 더 짧게 0.2초, ease-out으로 빠른 느낌을 줍니다                                              |
| 38 | <code>transition: width 1s ease-in-o...</code> | <code>width</code> 한 가지만 전환합니다. 1초로 길게 설정해 진행 상황이 느껴지게 합니다                                             |
| 42 | <code>.progress-bar:hover .progress-...</code> | 부모( <code>.progress-bar</code> )에 hover 시 자식( <code>.progress-fill</code> )의 <code>width</code> 를 바꿉니다 |

## transition-delay로 순차 등장 효과

`transition-delay` 를 활용해 목록 항목이 순서대로 나타나는 효과를 만듭니다.



```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>transition-delay</title>
  <style>
    .list {
      list-style: none;
      padding: 0;
    }

    .list-item {
      padding: 12px 16px;
      margin: 4px 0;
      background-color: #3b82f6;
      color: #fff;
      border-radius: 6px;
      opacity: 0;
      transform: translateX(-20px);
      transition:
        opacity 0.4s ease,
        transform 0.4s ease;
    }

    /* 컨테이너 hover 시 모든 항목에 적용 */
    .list:hover .list-item {
      opacity: 1;
      transform: translateX(0);
    }

    /* 각 항목에 지연 시간 개별 설정 */
    .list-item:nth-child(1) { transition-delay: 0s; }
    .list-item:nth-child(2) { transition-delay: 0.1s; }
    .list-item:nth-child(3) { transition-delay: 0.2s; }
    .list-item:nth-child(4) { transition-delay: 0.3s; }
  </style>
</head>
<body>
  <ul class="list">
    <li class="list-item">HTML 기초</li>
    <li class="list-item">CSS 레이아웃</li>
    <li class="list-item">JavaScript 입문</li>
    <li class="list-item">반응형 디자인</li>
  </ul>
</body>
</html>

```

## 브라우저 렌더링 결과

- 처음에는 목록 항목들이 투명하게 존재합니다.
- 마우스를 목록 위에 올리면 0.1초 간격으로 각 항목이 왼쪽에서 슬라이드되며 나타납니다.
- 마우스를 치우면 역방향으로 부드럽게 사라집니다.

### 줄별 코드 설명

| 줄     | 코드                                             | 설명                               |
|-------|------------------------------------------------|----------------------------------|
| 16    | <code>opacity: 0</code>                        | 초기 상태를 투명(보이지 않음)으로 설정합니다        |
| 17    | <code>transform: translateX(-20px)</code>      | 초기 위치를 왼쪽으로 20px 이동시킵니다          |
| 18    | <code>transition: opacity 0.4s ease,...</code> | 두 속성을 동일한 설정으로 전환합니다             |
| 25    | <code>opacity: 1; transform: transla...</code> | hover 시 완전히 불투명해지고 원래 위치로 이동합니다  |
| 29~32 | <code>transition-delay:</code>                 | 각 항목에 지연을 0.1초씩 증가시켜 순차적으로 나타냅니다 |

**팁:** `transition: all 0.3s ease` 처럼 `all` 을 사용하면 편리하지만, 예상치 못한 속성에도 전환이 적용되어 성능 문제가 생길 수 있습니다. 전환할 속성을 명시적으로 나열하는 것을 권장합니다.

## 절 요약

- `transition` 은 CSS 속성 값이 변할 때 그 중간 과정을 부드럽게 표현합니다.
- 네 가지 구성 요소: `property` , `duration` , `timing-function` , `delay`
- `ease` , `ease-out` 등 타이밍 함수로 가속·감속을 제어합니다.
- `display` 같이 보간 불가능한 속성에는 transition이 동작하지 않습니다.
- 여러 속성에 개별 설정이 필요할 때는 쉼표로 구분하여 나열합니다.

## 8.3 transform으로 요소 변형하기

### 도입

요소의 위치나 크기를 바꾸고 싶을 때 `width`, `margin`, `top` 같은 속성을 바꾸면 주변 레이아웃 전체가 다시 계산됩니다. 반면 **변형(transform)**은 레이아웃 흐름을 유지한 채 요소의 **시각적 표현**만 바꿉니다. 브라우저의 GPU를 활용하기 때문에 애니메이션 성능이 훨씬 우수합니다.

### 핵심 개념 설명

#### transform 함수 종류

##### transform

| 항목    | 내용                                                                                                                      |
|-------|-------------------------------------------------------------------------------------------------------------------------|
| 설명    | 요소에 2D 또는 3D 시각 변형을 적용하는 속성입니다. 레이아웃 흐름에 영향을 주지 않습니다                                                                    |
| 주요 함수 | <code>translate(x, y)</code> : X·Y 방향으로 이동. 단축형: <code>translateX(x)</code> , <code>translateY(y)</code>                |
|       | <code>rotate(각도)</code> : 시계 방향 회전. 단위는 <code>deg</code> (도) 또는 <code>turn</code> (바퀴 수)                                |
|       | <code>scale(배율)</code> : 크기 변경. <code>scale(1.5)</code> 는 1.5배 확대. <code>scaleX()</code> , <code>scaleY()</code> 단축형 가능 |
|       | <code>skew(x도, y도)</code> : 비틀기(기울이기). <code>skewX()</code> , <code>skewY()</code> 단축형 가능                               |
| 문법    | <code>transform: 함수1(값) 함수2(값)</code> (여러 함수를 공백으로 나열, 적용 순서 중요)                                                        |
| 사용 예시 | <code>transform: translateY(-4px) scale(1.05)</code>                                                                    |

#### transform-origin: 변형 기준점

**변형 기준점(transform-origin)**은 변형이 적용되는 중심점입니다. 기본값은 요소의 정중앙 ( 50% 50% )입니다.

```
/* 좌측 상단을 기준으로 회전 */  
transform-origin: top left;  
  
/* 키워드: top/center/bottom + left/center/right */  
transform-origin: bottom right;  
  
/* 퍼센트 또는 픽셀 */  
transform-origin: 20% 80%;  
transform-origin: 0 0;
```

## transform이 성능에 유리한 이유

브라우저의 렌더링 파이프라인은 크게 레이아웃(layout) → 페인트(paint) → 합성(composite) 단계로 이루어집니다.

- **width** , **margin** , **top** 변경: 레이아웃 단계부터 다시 계산 (비용 큼)
- **color** , **background-color** 변경: 페인트 단계부터 다시 계산 (중간 비용)
- **transform** , **opacity** 변경: 합성 단계만 실행, GPU에서 처리 (비용 매우 작음)

따라서 요소를 이동시킬 때는 **left** / **top** 대신 **transform: translate()** 를, 페이드 효과는 **visibility** 대신 **opacity** 를 사용하는 것이 성능에 유리합니다.

---

## 코드로 확인하기

### 네 가지 transform 함수 비교

각 transform 함수의 시각적 결과를 한 페이지에서 비교합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>transform 함수</title>
  <style>
    .container {
      display: flex;
      gap: 32px;
      padding: 40px;
      flex-wrap: wrap;
    }

    .box {
      width: 80px;
      height: 80px;
      background-color: #3b82f6;
      color: #fff;
      display: flex;
      align-items: center;
      justify-content: center;
      font-size: 0.75rem;
      border-radius: 4px;
      transition: transform 0.3s ease;
    }

    /* 이동: 오른쪽으로 20px, 아래로 20px */
    .translate:hover {
      transform: translate(20px, 20px);
    }

    /* 회전: 시계 방향 45도 */
    .rotate:hover {
      transform: rotate(45deg);
    }

    /* 확대: 1.3배 */
    .scale:hover {
      transform: scale(1.3);
    }

    /* 기울이기: X축 15도 */
    .skew:hover {
      transform: skewX(15deg);
    }

    /* 복합 변형 */
    .combo:hover {
      transform: rotate(10deg) scale(1.1) translateY(-8px);
    }
  </style>
</head>
</html>
```

```

    }
  </style>
</head>
<body>
  <div class="container">
    <div class="box translate">이동</div>
    <div class="box rotate">회전</div>
    <div class="box scale">확대</div>
    <div class="box skew">기울기</div>
    <div class="box combo">복합</div>
  </div>
</body>
</html>

```

## 브라우저 렌더링 결과

- 파란 박스 5개가 가로로 나열됩니다.
- 각 박스에 마우스를 올리면 이름에 맞는 변형이 0.3초 동안 부드럽게 적용됩니다.
- "이동" 박스는 오른쪽 아래로 슬라이드하고, "회전"은 45도 돌아가고, "확대"는 커집니다.
- 변형 중에도 주변 박스들의 위치는 변하지 않습니다.

### 줄별 코드 설명

| 줄  | 코드                                             | 설명                                                |
|----|------------------------------------------------|---------------------------------------------------|
| 27 | <code>transition: transform 0.3s ease</code>   | 모든 박스에 <code>transform</code> 속성의 변화를 0.3초로 전환합니다 |
| 31 | <code>transform: translate(20px, 20p...</code> | X 20px, Y 20px 이동. 원래 자리를 유지하면서 시각적 위치만 바꿉니다      |
| 36 | <code>transform: rotate(45deg)</code>          | 시계 방향 45도 회전. 기준점은 기본적으로 요소 중앙입니다                 |
| 41 | <code>transform: scale(1.3)</code>             | 가로·세로 모두 1.3배 확대합니다. 주변 레이아웃에는 영향을 주지 않습니다        |
| 46 | <code>transform: skewX(15deg)</code>           | X축 방향으로 15도 기울입니다. 평행사변형처럼 변형됩니다                  |
| 50 | <code>transform: rotate(10deg) scale...</code> | 여러 변형을 공백으로 나열합니다. 오른쪽에서 왼쪽 순서로 적용됩니다             |

## **transform-origin 활용**

기준점을 바꾸면 같은 회전도 다르게 보입니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>transform-origin</title>
  <style>
    .demo-wrap {
      display: flex;
      gap: 48px;
      padding: 48px;
    }

    .door {
      width: 60px;
      height: 100px;
      background-color: #f59e0b;
      border: 2px solid #d97706;
      border-radius: 2px 8px 8px 2px;
      transition: transform 0.5s ease-in-out;
    }

    /* 문 열리는 효과: 왼쪽 끝을 기준으로 회전 */
    .door-left {
      transform-origin: left center;
    }
    .door-left:hover {
      transform: rotateY(70deg); /* 3D 회전 */
    }

    /* 중앙 기준 뒤집기 */
    .door-center {
      transform-origin: center center; /* 기본값 */
    }
    .door-center:hover {
      transform: scaleX(-1);
    }

    /* 오른쪽 아래를 기준으로 회전 */
    .door-corner {
      transform-origin: bottom right;
    }
    .door-corner:hover {
      transform: rotate(-90deg);
    }
  </style>
</head>
<body>
  <div class="demo-wrap">
    <div class="door door-left">왼쪽</div>
```



```
<div class="door door-center">중앙</div>
<div class="door door-corner">모서리</div>
</div>
</body>
</html>
```

## 브라우저 렌더링 결과

- 세 개의 주황색 직사각형이 나란히 표시됩니다.
- 왼쪽: 문이 경첩처럼 왼쪽 끝을 축으로 열립니다.
- 중앙: 중앙을 기준으로 좌우가 뒤집힙니다.
- 오른쪽: 오른쪽 하단을 기준으로 접힙니다.

**베스트 프랙티스:** 카드 뒤집기, 아이콘 회전, 메뉴 열기/닫기 애니메이션에 transform을 적극 활용합니다. `left` / `top` 으로 위치를 바꾸는 것보다 GPU에서 처리되어 훨씬 부드럽습니다.

## 절 요약

- `transform` 은 `translate` , `rotate` , `scale` , `skew` 함수로 요소를 시각적으로 변형합니다.
- 레이아웃 재계산 없이 GPU에서 처리되어 성능이 우수합니다.
- `transform-origin` 으로 변형 기준점을 바꿀 수 있습니다. 기본값은 요소 중앙입니다.
- 여러 transform 함수를 공백으로 나열하면 오른쪽부터 왼쪽 순서로 적용됩니다.
- 이동은 `left` / `top` 대신 `transform: translate()` 를 사용합니다.

## 8.4 keyframes 애니메이션

### 도입

`transition` 은 A 상태에서 B 상태로의 한 번의 변화를 부드럽게 만들어 줍니다. 하지만 로딩 스피너처럼 반복되는 동작이나, 여러 단계를 거치는 복잡한 움직임을 만들려면 **키프레임 애니**

**메이션(keyframes animation)**이 필요합니다. 애니메이션의 타임라인을 퍼센트로 직접 정의할 수 있습니다.

## 핵심 개념 설명

### @keyframes 규칙

**@keyframes** 는 애니메이션의 각 시점에서 어떤 스타일이 적용될지를 정의합니다.

```
@keyframes 애니메이션이름 {  
  from { /* 시작 (0%와 동일) */ }  
  to   { /* 끝   (100%와 동일) */ }  
}  
  
/* 또는 퍼센트로 중간 단계 지정 */  
@keyframes 애니메이션이름 {  
  0%   { /* 시작 스타일 */ }  
  50%  { /* 중간 스타일 */ }  
  100% { /* 끝 스타일 */ }  
}
```

### animation 단축 속성

**animation**

| 항목    | 내용                                                                                                                             |
|-------|--------------------------------------------------------------------------------------------------------------------------------|
| 설명    | <code>@keyframes</code> 로 정의된 애니메이션을 요소에 적용하는 단축 속성입니다                                                                         |
| 구성 요소 | <code>animation-name</code> : 적용할 <code>@keyframes</code> 이름                                                                   |
|       | <code>animation-duration</code> : 1회 재생 시간 (예: <code>1s</code> , <code>500ms</code> )                                          |
|       | <code>animation-timing-function</code> : 속도 곡선 ( <code>ease</code> , <code>linear</code> 등)                                    |
|       | <code>animation-delay</code> : 시작 전 대기 시간                                                                                      |
|       | <code>animation-iteration-count</code> : 반복 횟수. <code>infinite</code> 로 무한 반복                                                  |
|       | <code>animation-direction</code> : 재생 방향. <code>normal</code> (앞으로), <code>reverse</code> (뒤로), <code>alternate</code> (앞뒤 교차) |
|       | <code>animation-fill-mode</code> : 애니메이션 전후 스타일. <code>forwards</code> (끝 상태 유지), <code>backwards</code> (시작 상태 유지)            |
| 문법    | <code>animation: name duration timing-function delay iteration-count direction fill-mode</code>                                |
| 사용 예시 | <code>animation: spin 1s linear infinite</code>                                                                                |

### animation-direction 비교

| 값                              | 설명                   | 사용 예          |
|--------------------------------|----------------------|---------------|
| <code>normal</code>            | 매번 0%에서 100%로 재생     | 기본값           |
| <code>reverse</code>           | 매번 100%에서 0%로 재생     | 역방향 이동        |
| <code>alternate</code>         | 홀수 회는 정방향, 짝수 회는 역방향 | 왕복 운동         |
| <code>alternate-reverse</code> | 홀수 회는 역방향, 짝수 회는 정방향 | 왕복 운동(역방향 시작) |

## 코드로 확인하기

### 로딩 스피너

`@keyframes` 로 무한 회전하는 로딩 스피너를 만듭니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>keyframes 애니메이션</title>
  <style>
    /* 로딩 스피너 */
    .spinner {
      width: 48px;
      height: 48px;
      border: 5px solid #e2e8f0;          /* 배경 원 */
      border-top-color: #3b82f6;         /* 파란 부분 */
      border-radius: 50%;
      animation: spin 0.8s linear infinite;
    }

    /* 스피너 회전 keyframes */
    @keyframes spin {
      from { transform: rotate(0deg); }
      to   { transform: rotate(360deg); }
    }

    /* 페이드 인/아웃 (alternate로 왕복) */
    .pulse {
      width: 16px;
      height: 16px;
      background-color: #10b981;
      border-radius: 50%;
      margin-top: 24px;
      animation: pulse-opacity 1.2s ease-in-out infinite alternate;
    }

    @keyframes pulse-opacity {
      from { opacity: 1; transform: scale(1); }
      to   { opacity: 0.3; transform: scale(0.8); }
    }
  </style>
</head>
<body>
  <div class="spinner" role="status" aria-label="로딩 중"></div>
  <div class="pulse"></div>
</body>
</html>

```

브라우저 렌더링 결과

- 흰 원형 테두리에 파란 부분이 0.8초 간격으로 계속 돌아가는 스피너가 표시됩니다.
- 그 아래 초록색 원이 커졌다 작아졌다를 반복하며 숨 쉬는 듯한 효과를 냅니다.

#### 줄별 코드 설명

| 줄     | 코드                                             | 설명                                                                         |
|-------|------------------------------------------------|----------------------------------------------------------------------------|
| 11    | <code>border-top-color: #3b82f6</code>         | 원형 테두리 중 위쪽만 파란색으로 바꿔 반원 효과를 만듭니다                                          |
| 12    | <code>animation: spin 0.8s linear in...</code> | <code>spin</code> 키프레임을 0.8초, 일정 속도 ( <code>linear</code> ), 무한 반복으로 실행합니다 |
| 15~18 | <code>@keyframes spin</code>                   | <code>from</code> (0deg)에서 <code>to</code> (360deg)로 360도 회전합니다            |
| 28    | <code>animation: pulse-opacity 1.2s ...</code> | <code>alternate</code> 로 홀수 번은 정방향, 짝수 번은 역방향으로 재생하여 왕복합니다                 |
| 31~34 | <code>@keyframes pulse-opacity</code>          | 시작(불투명.크기 1)과 끝(반투명.크기 0.8) 두 상태를 정의합니다                                    |

### 단계별 애니메이션: 알림 배지 등장

여러 퍼센트 지점을 사용하는 다단계 애니메이션을 만듭니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>다단계 애니메이션</title>
  <style>
    .notification {
      display: inline-flex;
      align-items: center;
      gap: 8px;
      padding: 12px 20px;
      background-color: #fff;
      border: 1px solid #e2e8f0;
      border-radius: 12px;
      box-shadow: 0 4px 12px rgba(0,0,0,0.1);

      /* 페이지 로드 시 자동 실행, 끝 상태 유지 */
      animation: slide-in 0.6s cubic-bezier(0.34, 1.56, 0.64, 1) forwards;
      opacity: 0; /* 초기 상태: 투명 */
    }

    /* cubic-bezier(0.34, 1.56, 0.64, 1) = 살짝 튕기는 spring 효과 */
    @keyframes slide-in {
      0% {
        opacity: 0;
        transform: translateY(20px) scale(0.9);
      }
      60% {
        opacity: 1;
        transform: translateY(-4px) scale(1.02); /* 약간 위로 */
      }
      100% {
        opacity: 1;
        transform: translateY(0) scale(1);
      }
    }

    .badge {
      background-color: #ef4444;
      color: #fff;
      font-size: 0.75rem;
      font-weight: bold;
      padding: 2px 7px;
      border-radius: 999px;
      animation: bounce 0.8s 0.6s ease both; /* 0.6s 딜레이로 알림 등장 후 시작 */
    }

    @keyframes bounce {
      0%, 100% { transform: scale(1); }
  
```

```

    40%      { transform: scale(1.4); }
    60%      { transform: scale(0.9); }
  }
</style>
</head>
<body>
  <div class="notification">
    새 메시지가 도착했습니다
    <span class="badge">3</span>
  </div>
</body>
</html>

```

## 브라우저 렌더링 결과

1. 페이지 로드 시: 알림 카드가 아래쪽에서 살짝 튀어 오르며 나타납니다.
2. 0.6초 뒤: 빨간 배지 숫자가 부풀어 오르는 바운스 효과와 함께 등장합니다.
3. 최종 상태가 유지됩니다( `animation-fill-mode: forwards` ).

### 줄별 코드 설명

| 줄     | 코드                                             | 설명                                                                                                     |
|-------|------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 15    | <code>animation: slide-in 0.6s cubic...</code> | 커스텀 베지어 곡선으로 스프링 효과를 만듭니다. <code>forwards</code> 는 끝 스타일(100%)을 유지합니다                                  |
| 16    | <code>opacity: 0</code>                        | 초기 상태를 투명으로 설정합니다.<br><code>animation-fill-mode: backwards</code> 도 설정된 것처럼 동작합니다                      |
| 21~30 | <code>@keyframes slide-in</code>               | 0%→60%→100% 세 지점을 정의합니다.<br>60% 지점에서 목표치를 살짝 넘겨<br>(overshoot) 자연스러운 느낌을 줍니다                           |
| 35    | <code>animation: bounce 0.8s 0.6s ea...</code> | 지속시간 0.8s, 딜레이 0.6s입니다. <code>both</code> 는 <code>forwards</code> 와 <code>backwards</code> 를 동시에 적용합니다 |

**주의:** 과도한 애니메이션은 사용자 경험을 저해합니다. 특히 전정기관 장애가 있는 사용자는 큰 움직임에 불편함을 느낄 수 있습니다. CSS 미디어 쿼리 `@media (prefers-`



`reduced-motion: reduce`) 로 움직임 감소 설정을 존중하는 코드를 추가하는 것을 권장합니다.

```
@media (prefers-reduced-motion: reduce) {  
  .spinner,  
  .pulse,  
  .notification,  
  .badge {  
    animation: none;  
  }  
}
```

## 절 요약

- `@keyframes` 는 퍼센트 또는 `from / to` 로 애니메이션의 타임라인을 정의합니다.
- `animation` 속성으로 이름, 시간, 타이밍, 반복, 방향, 채우기 모드를 설정합니다.
- `infinite` 로 무한 반복, `alternate` 로 왕복 운동을 만들 수 있습니다.
- `animation-fill-mode: forwards` 는 애니메이션 종료 후 끝 상태를 유지합니다.
- `prefers-reduced-motion` 으로 움직임에 민감한 사용자를 배려합니다.

## 8.5 CSS 변수로 테마 관리하기

### 도입

웹 사이트의 브랜드 색상이 바뀌면 CSS 파일 전체를 검색해서 색상값을 하나하나 수정해야 할까요? **CSS 변수(커스텀 속성, CSS Custom Properties)**를 사용하면 값을 한 곳에서 선언하고 필요한 곳마다 참조할 수 있습니다. Sass 같은 전처리기가 없어도 순수 CSS만으로 디자인 토큰을 관리하고 다크 모드 테마를 구현할 수 있습니다.

# 핵심 개념 설명

## CSS 변수 (CSS Custom Properties)

CSS 변수는 `--` 로 시작하는 이름으로 값을 선언하고 `var()` 함수로 불러오는 사용자 정의 속성입니다.

```
/* 선언: -- + 이름 */
--primary-color: #3b82f6;

/* 사용: var(변수명, 기본값) */
color: var(--primary-color);
color: var(--primary-color, #000); /* 두 번째 인자는 변수가 없을 때의 기본값 */
```

### var()

| 항목    | 내용                                                                 |
|-------|--------------------------------------------------------------------|
| 설명    | CSS 커스텀 속성(변수) 값을 가져오는 함수입니다                                       |
| 매개변수  | <code>--custom-property-name</code> (필수): 사용할 커스텀 속성 이름            |
|       | <code>fallback-value</code> (선택): 변수가 정의되지 않았거나 유효하지 않을 때 사용할 대체 값 |
| 반환값   | 해당 커스텀 속성에 설정된 값                                                   |
| 사용 예시 | <code>color: var(--text-color, #333)</code>                        |

## :root와 변수 범위

CSS 변수는 선언된 요소의 자손에게만 유효합니다. 페이지 전역에서 사용할 변수는 최상위 선택자인 `:root` 에 선언합니다. `:root` 는 `<html>` 요소와 동일하지만 명시도가 더 높습니다.

```
:root {
  --color-primary: #3b82f6;
  --color-bg: #ffffff;
  --spacing-md: 16px;
  --radius-default: 8px;
}
```

## Sass 변수와의 차이점

| 비교 항목     | CSS 변수                    | Sass 변수             |
|-----------|---------------------------|---------------------|
| 런타임 변경    | 가능 (JavaScript로 동적 변경 가능) | 불가능 (컴파일 시 고정)      |
| 범위(scope) | 상속 기반, 하위 요소에서 재정의 가능     | 파일 범위               |
| 브라우저 지원   | IE11 제외 모든 현대 브라우저        | 컴파일 후 일반 CSS로 변환    |
| 사용 방법     | <code>var(--name)</code>  | <code>\$name</code> |

CSS 변수가 Sass 변수와 가장 다른 점은 **런타임에 동적으로 바뀔 수 있다**는 것입니다.

JavaScript에서 `element.style.setProperty('--primary', '#ff0000')` 으로 즉시 변경할 수 있고, 이 변화가 해당 변수를 사용하는 모든 속성에 자동으로 반영됩니다.

## 코드로 확인하기

### CSS 변수로 디자인 시스템 구축

`:root` 에 변수를 선언하고 전체 컴포넌트에서 일관되게 사용합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS 변수 기초</title>
  <style>
    /* 전역 디자인 토큰 */
    :root {
      --color-primary: #3b82f6;
      --color-primary-dk: #2563eb;
      --color-danger: #ef4444;
      --color-success: #10b981;
      --color-text: #1e293b;
      --color-bg: #f8fafc;
      --color-surface: #ffffff;
      --color-border: #e2e8f0;

      --spacing-sm: 8px;
      --spacing-md: 16px;
      --spacing-lg: 24px;

      --radius-sm: 4px;
      --radius-md: 8px;
      --radius-full: 9999px;

      --shadow-md: 0 4px 12px rgba(0, 0, 0, 0.08);
    }

    body {
      background-color: var(--color-bg);
      color: var(--color-text);
      font-family: sans-serif;
      padding: var(--spacing-lg);
    }

    .card {
      background-color: var(--color-surface);
      border: 1px solid var(--color-border);
      border-radius: var(--radius-md);
      padding: var(--spacing-lg);
      box-shadow: var(--shadow-md);
      max-width: 320px;
    }

    .btn {
      display: inline-block;
      padding: var(--spacing-sm) var(--spacing-md);
      background-color: var(--color-primary);
      color: #fff;
```

```

    border: none;
    border-radius: var(--radius-sm);
    cursor: pointer;
    transition: background-color 0.2s ease;
  }

  .btn:hover {
    background-color: var(--color-primary-dk);
  }

  /* 변형: danger 버튼은 변수 하나만 재정의 */
  .btn-danger {
    background-color: var(--color-danger);
  }

  .badge {
    display: inline-block;
    padding: 2px var(--spacing-sm);
    background-color: var(--color-success);
    color: #fff;
    border-radius: var(--radius-full);
    font-size: 0.75rem;
  }
</style>
</head>
<body>
  <div class="card">
    <h2>CSS 변수 예제</h2>
    <p>디자인 토큰을 <code>:root</code>에 선언하면 한 곳에서 관리합니다.</p>
    <span class="badge">신규</span>
    <br><br>
    <button class="btn">기본 버튼</button>
    <button class="btn btn-danger">삭제</button>
  </div>
</body>
</html>

```

## 브라우저 렌더링 결과

- 연한 회색 배경에 흰색 카드가 표시됩니다.
- 파란 기본 버튼과 빨간 삭제 버튼이 나란히 놓입니다.
- 기본 버튼에 hover 시 더 진한 파란색으로 바뀝니다.
- 색상, 간격, 모서리 반지름이 모두 `:root` 변수에서 일관되게 관리됩니다.

### 줄별 코드 설명

| 줄    | 코드                                             | 설명                                                                                                       |
|------|------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| 6~24 | <code>:root { --변수명: 값; }</code>               | 전역 커스텀 속성을 선언합니다. 명명 규칙은 자유롭지만 <code>--color-</code> , <code>--spacing-</code> 같이 카테고리 접두사를 붙이면 관리가 편합니다 |
| 27   | <code>background-color: var(--color-...</code> | 선언된 변수를 불러옵니다. 나중에 <code>--color-bg</code> 값만 바꾸면 이 속성도 자동으로 바뀝니다                                        |
| 60   | <code>.btn-danger { background-color...</code> | <code>.btn</code> 의 나머지 스타일은 그대로 사용하고 배경색 변수만 재정의합니다. 코드 중복 없이 변형을 만드는 패턴입니다                             |

## CSS 변수로 다크 모드 테마 구현

미디어 쿼리와 CSS 변수를 조합하여 시스템 다크 모드에 자동 대응하는 테마를 만듭니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>다크 모드 테마</title>
  <style>
    /* 라이트 모드 기본값 */
    :root {
      --color-bg: #ffffff;
      --color-surface: #f8fafc;
      --color-text: #1e293b;
      --color-text-sub: #64748b;
      --color-border: #e2e8f0;
      --color-primary: #3b82f6;
    }

    /* 시스템 다크 모드 감지 시 변수 재정의 */
    @media (prefers-color-scheme: dark) {
      :root {
        --color-bg: #0f172a;
        --color-surface: #1e293b;
        --color-text: #f1f5f9;
        --color-text-sub: #94a3b8;
        --color-border: #334155;
        --color-primary: #60a5fa;
      }
    }

    /* 모든 컴포넌트는 변수만 참조 → 테마에 자동 대응 */
    body {
      background-color: var(--color-bg);
      color: var(--color-text);
      font-family: sans-serif;
      padding: 24px;
      transition: background-color 0.3s ease, color 0.3s ease;
    }

    .card {
      background-color: var(--color-surface);
      border: 1px solid var(--color-border);
      border-radius: 12px;
      padding: 24px;
      max-width: 360px;
    }

    .card h2 {
      color: var(--color-text);
      margin-top: 0;
    }
  </style>

```

```

    .card p {
      color: var(--color-text-sub);
      line-height: 1.6;
    }

    .link {
      color: var(--color-primary);
      text-decoration: none;
    }

    .link:hover {
      text-decoration: underline;
    }
  </style>
</head>
<body>
  <div class="card">
    <h2>테마 자동 전환</h2>
    <p>
      시스템 설정에 따라 라이트/다크 모드가 자동으로 바뀝니다.
      <a class="link" href="#">더 알아보기</a>
    </p>
  </div>
</body>
</html>

```

## 브라우저 렌더링 결과

- 라이트 모드: 흰 배경에 짙은 텍스트와 연한 회색 카드가 표시됩니다.
- 다크 모드(시스템 설정에서 활성화): 어두운 남색 배경으로 부드럽게 전환됩니다.
- 텍스트, 테두리, 링크 색상이 모두 다크 모드에 맞게 자동으로 바뀝니다.

### 줄별 코드 설명



| 줄     | 코드                                               | 설명                                                  |
|-------|--------------------------------------------------|-----------------------------------------------------|
| 6~13  | <code>:root { ... }</code>                       | 라이트 모드의 기본 색상 변수를 선언합니다                             |
| 16    | <code>@media (prefers-color-scheme: ...</code>   | 운영체제의 다크 모드 설정을 감지합니다. ch_07에서 배운 미디어 쿼리를 테마에 활용합니다 |
| 17~24 | <code>:root { ... }</code> 재정의                   | 다크 모드 조건에서 같은 변수들을 어두운 값으로 덮어씁니다                    |
| 28~29 | <code>background-color: var( ... ); co...</code> | 컴포넌트 코드는 변수만 참조하므로 변수 값만 바뀌어도 자동으로 테마가 적용됩니다        |
| 30    | <code>transition: background-color 0...</code>   | 테마 전환 시 부드럽게 색상이 바뀝니다                               |

**팁:** CSS 변수와 JavaScript를 연동하면 버튼 클릭으로 다크 모드를 수동 전환하는 기능을 구현할 수 있습니다. `document.documentElement.style.setProperty('--color-bg', '#000')` 처럼 `:root`의 변수를 JavaScript로 직접 바꾸면 됩니다.

**베스트 프랙티스:** 색상값을 하드코딩 하지 말고 CSS 변수로 관리합니다. 변수 이름은 의미( `--color-primary` )를 사용하고 색상 그대로의 이름( `--blue` )은 피합니다. 테마가 바뀌면 이름이 의미를 잃기 때문입니다.

## 절 요약

- CSS 변수는 `--이름: 값` 으로 선언하고 `var(--이름)` 으로 사용합니다.
- `:root` 에 선언하면 페이지 전역에서 사용할 수 있습니다.
- `var()` 의 두 번째 인자로 기본값을 지정할 수 있습니다.
- `@media (prefers-color-scheme: dark)` 와 결합하면 시스템 다크 모드에 자동 대응합니다.
- CSS 변수는 런타임에 JavaScript로 동적 변경이 가능합니다.

## FAQ

### Q1: transition이 작동하지 않을 때 가장 먼저 확인해야 할 것은 무엇입니까?

A1: 세 가지를 순서대로 확인합니다. 1. `transition-duration` 이 `0s` 가 아닌지 확인합니다. duration을 생략하면 기본값이 `0s` 라 즉시 바뀝니다. 2. 전환하려는 속성이 보간 가능한지 확인합니다. `display: none` 에서 `display: block` 으로의 변화는 보간이 불가능합니다. 이 경우 `opacity` 와 `visibility` 를 함께 사용하거나, `height: 0` 에서 `height: auto` 대신 `max-height` 를 활용합니다. 3. `transition` 을 초기 상태에 선언했는지 확인합니다. `:hover` 에만 선언하면 hover 진입 시에만 전환이 적용되고 나갈 때는 즉시 바뀝니다.

### Q2: `::before` 와 `::after` 로 만든 내용은 화면에 보이지 않을 때 무엇을 확인해야 합니까?

A2: `content` 속성이 반드시 있어야 합니다. 값이 `""` 빈 문자열이어도 됩니다. 내용은 있지만 안 보인다면 `position: absolute` 를 사용할 때 부모 요소에 `position: relative` 가 없어서 보이는 영역 바깥에 위치했을 가능성이 있습니다. 브라우저 개발자 도구에서 가상 요소도 DOM 트리에 표시되므로 직접 확인할 수 있습니다.

### Q3: CSS 변수와 Sass 변수의 차이는 무엇입니까?

A3: 가장 큰 차이는 런타임 변경 가능 여부입니다. Sass 변수( `$primary: blue` )는 CSS로 컴파일될 때 값이 고정되어 브라우저는 변수를 모릅니다. CSS 변수( `--primary: blue` )는 브라우저가 직접 이해하며 JavaScript로 실시간 변경이 가능합니다. 또한 CSS 변수는 상속되므로 하위 요소에서 같은 이름으로 재선언하면 그 범위 안에서만 덮어쓸 수 있습니다. Sass 빌드 환경이 없는 순수 CSS 프로젝트에서는 CSS 변수만으로도 충분히 테마 관리가 가능합니다.

### Q4: transform 속성을 여러 개 나열할 때 순서가 중요합니까?

A4: 매우 중요합니다. `transform: translateX(50px) rotate(45deg)` 와 `transform: rotate(45deg) translateX(50px)` 는 다른 결과를 냅니다. CSS transform은 오른쪽에 나열된 함수부터 먼저 적용됩니다. 회전 후 이동하면 회전된 좌표계 기준으로 이동하기 때문에 결과가 달라집니다. 일반적으로 직관적인 결과를 얻으려면 `translate` → `rotate` → `scale` 순서를 사용합니다.

### Q5: 애니메이션이 처음에는 잘 보이다가 갑자기 초기 위치로 돌아가는 현상은 왜 발생합니까?

A5: `animation-fill-mode` 가 설정되지 않아서입니다. 기본값은 `none` 으로, 애니메이션이 끝나면 적용 전 상태로 돌아갑니다. 끝 상태를 유지하려면 `animation-fill-mode: forwards`

를 추가합니다. 만약 지연( `animation-delay` )이 있고 시작 전에도 시작 상태를 유지하려면 `backwards` 를, 둘 다 원하면 `both` 를 사용합니다.

---

## 장 요약

이번 장에서는 CSS의 인터랙션과 시각 효과를 담당하는 핵심 기능들을 배웠습니다. 의사 클래스로 요소의 상태에 반응하고, `::before` 와 `::after` 로 HTML 수정 없이 장식을 추가할 수 있습니다. `transition` 은 속성 값 변화를 부드럽게, `transform` 은 레이아웃을 건드리지 않고 요소를 이동·회전·확대합니다. `@keyframes` 로 반복과 다단계 애니메이션을 정의하고, CSS 변수로 디자인 토큰을 한 곳에서 관리하여 다크 모드 같은 테마 전환을 우아하게 구현할 수 있습니다.

## 핵심 키워드

의사 클래스/요소, `transition`, `transform`, `@keyframes` 애니메이션, CSS 변수

## 다음 장 미리보기

9장부터는 JavaScript의 세계로 들어갑니다. 변수 선언, 자료형, 조건문, 반복문 등 프로그래밍의 기본 구조를 배우면서 CSS로만 표현할 수 없었던 동적 동작을 직접 구현하게 됩니다.

---

## ✂ 실습 프로젝트 (Hands-on Project)

### 8장 실습 프로젝트: 인터랙티브 애니메이션 버튼 모음

---

이 장에서 배운 개념을 실무 시나리오에 적용하는 프로젝트입니다.

---

#### 초급: 인터랙티브 버튼 컬렉션 페이지

##### 시나리오

포트폴리오 사이트에 들어갈 버튼 컴포넌트 페이지를 만들고 있습니다. 기획자는 "버튼 위에 마우스를 올렸을 때 단순히 색만 바뀌는 것이 아니라 움직임이 느껴지면

좋겠다"고 요청했습니다. transition과 transform을 활용하여 세 종류의 인터랙티브 버튼을 완성하세요.

### 학습 목표

- `:hover` / `:focus` / `:active` 의사 클래스로 버튼 상태를 스타일링합니다.
- `transition` 으로 부드러운 상태 전환 효과를 구현합니다.
- `transform` 으로 레이아웃 영향 없이 시각 변형을 적용합니다.

### 진행 방법

1. `project_starter.html` 을 열고 `TODO` 주석을 찾습니다.
2. 각 `TODO` 를 이 장에서 배운 개념으로 완성합니다.
3. 완성 후 브라우저에서 열어 세 버튼의 `hover` 효과를 확인합니다.

### 완성 기준

- `[] .btn-lift: hover` 시 위로 4px 이동 + 그림자 등장 (0.25s ease)
- `[] .btn-fill: hover` 시 배경이 왼쪽에서 오른쪽으로 채워지는 효과 (`::before + scaleX`)
- `[] .btn-pulse: 클릭(active)` 시 살짝 작아지는 효과 (scale 0.95)
- `[]` 모든 버튼에 `:focus` 스타일이 있어야 합니다 (접근성)
- `[] prefers-reduced-motion` 처리가 포함되어야 합니다

### 실행 방법

브라우저에서 `project_starter.html` 파일을 열어 확인합니다.

## 중급: CSS 변수 기반 테마 전환 + 키프레임 컴포넌트 페이지

### 시나리오

SaaS 대시보드의 상태 알림 영역을 개발하고 있습니다. 페이지에는 라이트/다크 모드 전환 토글 버튼이 있고, 로딩 중에는 스피너와 스켈레톤 UI가 표시되어야 합니다. CSS 변수로 테마를 관리하고, JavaScript로 클래스를 토글하여 수동 다크 모드 전환을 구현하세요.

## 학습 목표

- CSS 변수로 라이트/다크 두 가지 테마를 관리합니다.
- JavaScript에서 `element.style.setProperty()` 또는 클래스 전환으로 테마를 동적으로 바꿉니다.
- `@keyframes` 로 로딩 스피너와 스켈레톤 shimmer 효과를 구현합니다.
- `prefers-color-scheme` (시스템 설정) + 수동 토글이 공존하는 패턴을 익힙니다.

## 진행 방법

1. `project_advanced_starter.html` 을 열고 `TODO` 주석을 찾습니다.
2. CSS 변수 선언과 다크 테마 재정의의 완성합니다.
3. `toggleTheme()` 함수를 구현하여 버튼 클릭 시 테마가 전환되도록 합니다.
4. 스피너와 스켈레톤 shimmer 의 `@keyframes` 를 작성합니다.

## 완성 기준

- `[]:root` 에 라이트 모드 변수가 선언되어 있습니다.
- `[] .dark` 클래스가 `html` 에 추가되면 다크 테마 변수로 전환됩니다.
- `[]` 토글 버튼 클릭 시 `html` 에 `.dark` 클래스가 토글됩니다.
- `[]` 로딩 스피너(`@keyframes spin`)가 작동합니다.
- `[]` 스켈레톤 카드에 shimmer 효과(`@keyframes shimmer`)가 적용됩니다.
- `[] prefers-reduced-motion` 처리가 포함됩니다.

## 실행 방법

브라우저에서 `project_advanced_starter.html` 파일을 열어 확인합니다.

## 확장 아이디어

- `LocalStorage` 에 테마 설정을 저장하여 페이지 새로고침 후에도 유지되도록 구현해 보세요.
- 시스템 다크 모드와 수동 설정이 충돌할 때 어느 쪽을 우선할지 정책을 정하고 구현해 보세요.

- 알림 카드(notification)가 화면 밖에서 슬라이드인 되는 애니메이션을 추가해 보세요.

▶  project\_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project\_complete.html (완성 코드)

▶  project\_advanced\_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project\_advanced\_complete.html (완성 코드)